

Chapter 10 Summary — Population-Based Training and Evolutionary Game Theory

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

10 Chapter 10 — Population-Based Training and Evolutionary Game Theory	1
10.1 Why populations — and why structure decides everything	2
10.2 The family of methods	2
10.3 Replicator dynamics — where a population can rest	3
10.4 The spinning top — transitive vs cyclic structure	6
10.5 The AlphaStar-style PBT league	9
10.6 Diversity — is the population actually diverse?	12
10.7 EGTA — evaluating the population, and a surprise	13
10.8 Honest notes, limitations, and where this hands off	15
10.9 Key takeaways for the thesis synthesis	16

10 Chapter 10 — Population-Based Training and Evolutionary Game Theory

This is a ground-up chapter on training and evaluating **populations** of agents: the evolutionary mathematics that says whether a population can settle at all, the transitive/cyclic structure that decides whether self-training converges or spins, and a small AlphaStar-style **league** built on a solvable poker game. It is written to be read on its own. **All experimental numbers reported here were measured** on reproducible runs and, wherever possible, are bounded by *exact* references (analytic ESS/Nash for the matrix games; Chapter 7’s exact best-response exploitability for Leduc). Where a run contradicted what theory led me to expect, I keep the original expectation and reconcile it with what happened — those gaps are the most instructive parts of the chapter.

Where this sits in the thesis. Chapters 2-8 solved *one* fixed game; Chapter 9 added a *second* learner. Chapter 10 goes to a *whole population* that trains against itself. It carries three thesis hooks. **Main exploiters** are automated opponent modelers — the population lift of Chapter 7’s static read (Contribution #1). The **AlphaStar league** uses exploiters to make its main agents robust — close to a population-level safe-exploitation mechanism, but without a formal guarantee: its authors motivate it by fictitious self-play, whose mixture converges to a Nash equilibrium in two-player zero-sum games, and support the league’s

robustness only empirically.¹ That is exactly where Contribution #2 lives. And **EGTA** / **meta-Nash** is the population evaluation methodology (Contribution #3).

10.1 Why populations — and why structure decides everything

Self-play has a famous failure mode: a player that only ever trains against its current self can go around in circles. If the game is **rock-paper-scissors**, “get better” has no meaning — Rock beats Scissors, loses to Paper, and improving against one opponent just makes you worse against another. The single most important idea in this chapter is that games have two kinds of structure mixed together: a **transitive** part (a genuine skill ladder — there *is* a better player) and a **cyclic** part (a wheel of counters — there is only *what beats what*). Whether a self-training population converges to something good or spins forever is decided by *which part dominates*, not by the learning rate.

A picture to hold onto: a **dojo**. The **main** students are what you are trying to produce. You keep a room full of **sparring partners** whose entire job is to find and punish a specific weakness in the current students (exploiters). And you keep a **museum of past champions** — frozen snapshots — so nobody wins by forgetting how to beat an old style. Training is then a loop: students spar, the strong ones get copied and slightly mutated (population-based training), and periodically a copy of everyone is frozen into the museum. The league is that dojo, made mathematical.¹

10.2 The family of methods

Every method here is a different way to turn “a population” into a training signal or an evaluation.

Table 1: The population-based method families: what each does, when to reach for it, and its main weakness.

Approach	What it does	Reach for it when	Main weakness
Self-play	train the latest policy against (a copy of) itself	transitive games; a quick baseline	cycles / forgets in cyclic games; last iterate does not converge
PSRO	population + meta-Nash + best-response oracle; add the BR, repeat	competitive games; want a game-theoretic target	a full best response per round; the <i>mixture</i> is not the answer (see §10.7)

¹Vinyals, O. et al. (2019). “Grandmaster level in StarCraft II using multi-agent reinforcement learning.” *Nature* 575(7782), 350–354. DOI 10.1038/s41586-019-1724-z (AlphaStar; the league and PFSP).

Approach	What it does	Reach for it when	Main weakness
PBT	train many agents; copy the fitter ones (exploit) + perturb hyper-parameters (explore)	any population; online hyper-parameter search	can collapse diversity or regress (see §10.5)
AlphaStar league	PBT with three agent <i>types</i> (main / main-exploiter / league-exploiter) + freezing + PFSP	non-transitive games where naive self-play cycles	many agents, much compute; heuristic, no safety guarantee

Two evaluation tools cut across them. **Replicator dynamics** (§10.3) are the continuous idealization of selection — they say where a population *flows* and where it can *rest*. The **spinning-top decomposition** (§10.4) measures how transitive vs cyclic a game (or a population) is. Together they are the diagnostic that predicts, *before* you spend the compute, whether the league in §10.5 will settle or spin.

Historically the line runs: population-based training as online hyper-parameter search (PBT, 2017)² → game theory over a policy population (PSRO, 2017)³ → the three-type league that reached Grandmaster level in StarCraft II, above 99.8% of ranked human players (AlphaStar, 2019)¹ → the geometry that explains *why* you need a population (transitive/cyclic decomposition, 2018-2019; spinning top, 2020).⁴ Such populations are evaluated with empirical game-theoretic analysis (EGTA).⁵⁶

10.3 Replicator dynamics — where a population can rest

The **replicator equation** is the simplest model of selection: a strategy’s share grows in proportion to how much it beats the *population average*,

$$\dot{x}_i = x_i [f_i(x) - \bar{f}(x)], \quad \bar{f}(x) = \sum_j x_j f_j(x).$$

²Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W. M. et al. (2017). “Population Based Training of Neural Networks.” *arXiv:1711.09846*.

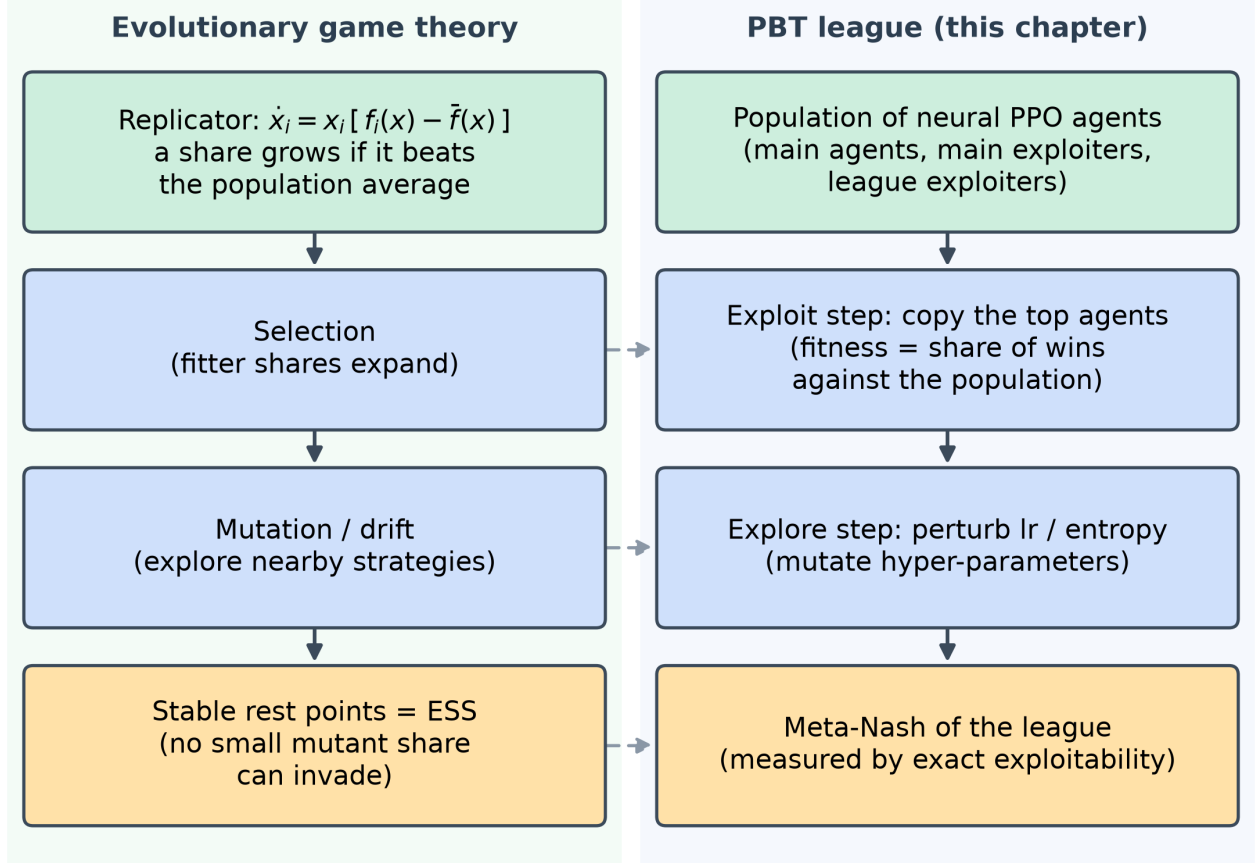
³Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO). See also: McMahan, H. B., Gordon, G. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML*, 536–543 (the double-oracle method PSRO generalizes).

⁴Czarnecki, W. M., Gidel, G., Tracey, B., Tuyls, K., Omidshafiei, S., Balduzzi, D. & Jaderberg, M. (2020). “Real World Games Look Like Spinning Tops.” *NeurIPS*; arXiv:2004.09468 — the spinning-top geometry and why training needs populations.

⁵Wellman, M. P. (2006). “Methods for Empirical Game-Theoretic Analysis.” *AAAI*, 1552–1555.

⁶Tuyls, K., Pérolat, J., Lanctot, M., Hughes, E., Everett, R., Leibo, J. Z., Szepesvári, C. & Graepel, T. (2020). “Bounds and dynamics for empirical game theoretic analysis.” *Autonomous Agents and Multi-Agent Systems* 34(1), 7. DOI 10.1007/s10458-019-09432-y.

Every Nash equilibrium is a rest point, but not conversely (every pure strategy is one too); stable rest points are Nash equilibria, and every **evolutionarily stable strategy (ESS)** — a pure or mixed strategy that, once adopted by the whole population, cannot be invaded by a small mutant share — is an asymptotically stable rest point.⁷ This is the continuous-time idealization of what a PBT league does discretely: copy the fitter agents (selection) and perturb them (mutation).



dashed = "discrete analog of": the league is replicator dynamics with learned agents instead of fixed strategy shares

Figure 1: Evolutionary game theory as the continuous idealization of a PBT league: replicator selection $\leftrightarrow$ copying the fitter agents, mutation/drift $\leftrightarrow$ perturbing learning rate and entropy, and stable rest points (ESS) $\leftrightarrow$ the meta-Nash mixture the league is trying to reach. The league is replicator dynamics with learned agents instead of fixed strategy shares.

Run on four canonical symmetric games (deterministic dynamics, so the outcome is unambiguous), the measured results match theory exactly:

⁷Hofbauer, J. & Sigmund, K. (2003). "Evolutionary game dynamics." *Bulletin of the American Mathematical Society* 40(4), 479–519 — the "folk theorem" of replicator dynamics (§2.3), asymptotic stability of an ESS (§2.6), and the closed orbits of rock-paper-scissors (Theorem 2).

Table 2: Replicator dynamics on four symmetric games, against the analytic reference.

Game	Analytic reference	Measured final	Converged?
Prisoner’s Dilemma	Defect dominates; ESS (0, 1)	[0.0, 1.0]	yes
Hawk-Dove	interior ESS $p(\text{Hawk}) = V/C = 0.5$	[0.5, 0.5] (orbit radius 0.0)	yes
Rock-Paper-Scissors	Nash = uniform; no ESS	orbits centre, radius 0.095	no
Stag Hunt	two pure ESS (basin-dependent)	$[0.8, 0.2] \rightarrow [1, 0];$ $[0.2, 0.8] \rightarrow [0, 1]$	yes

Prisoner’s Dilemma collapses to the dominant strategy; Hawk-Dove *reaches* the interior 0.5 ESS; Stag Hunt picks a different pure ESS depending on the basin its start lands in. **Rock-Paper-Scissors never converges** — its interior fixed point is a *centre*, so the population orbits it forever (closed in continuous time; explicit Euler turns them into a slowly widening spiral). That last row is the whole motivation for the rest of the chapter: a cyclic game has no stable population.⁸

⁸Hofbauer, J. & Sigmund, K. (1998). *Evolutionary Games and Population Dynamics* (Cambridge) — replicator dynamics, ESS, and the RPS centre; see also the Bloembergen, Tuyls, Hennes & Kaisers survey (JAIR 53, 659–697, 2015) connecting replicator dynamics to multi-agent learning.

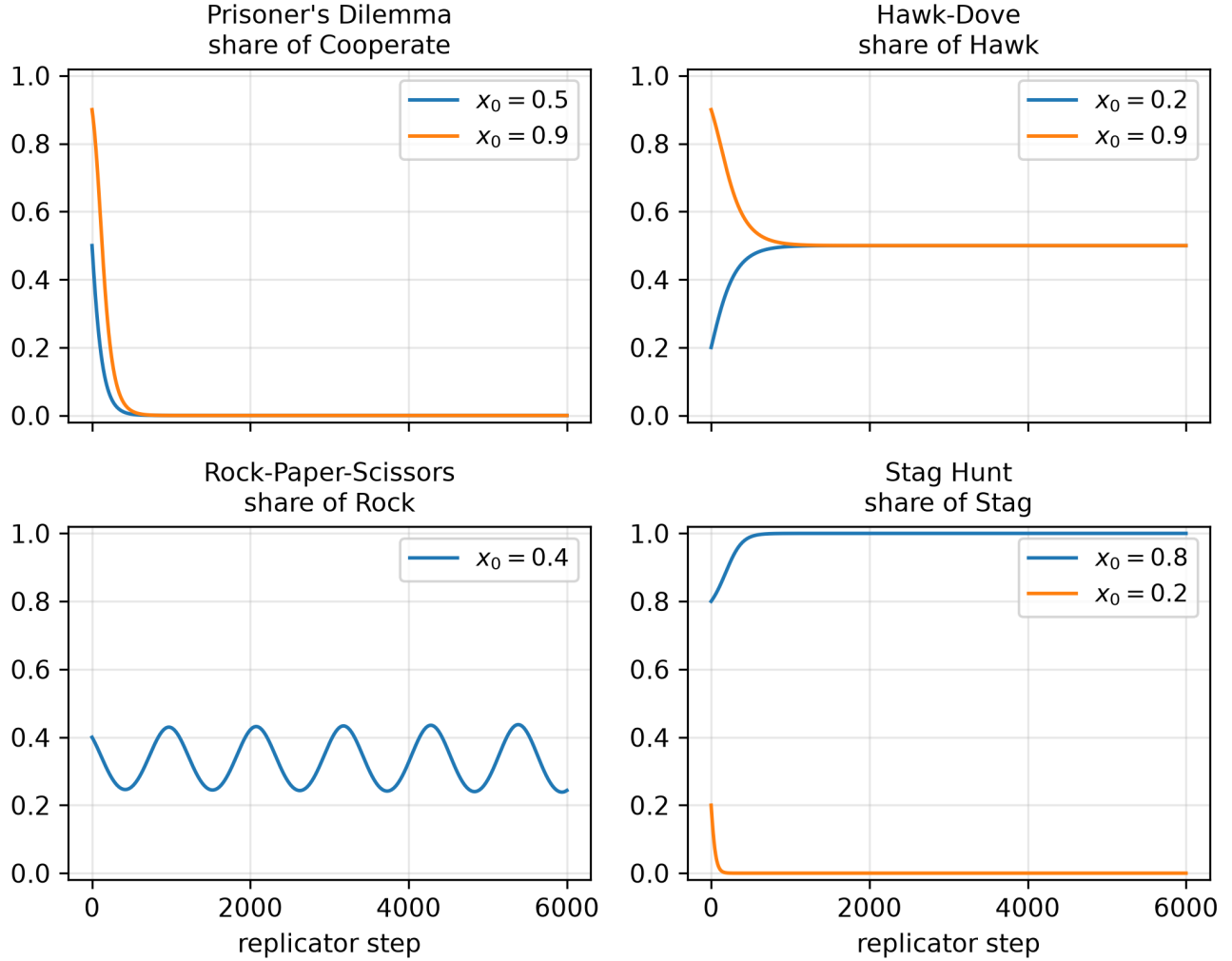
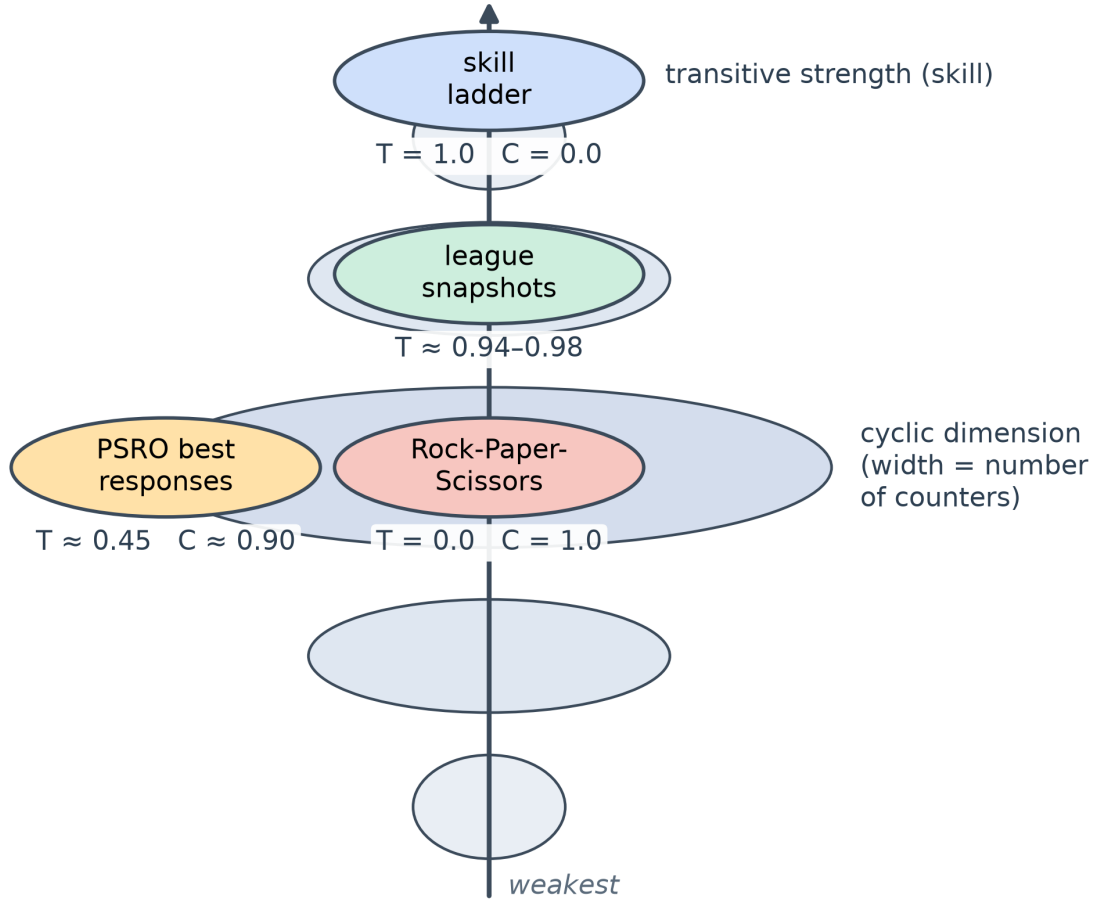


Figure 2: Replicator trajectories (share of the first strategy over time): Prisoner’s Dilemma → all-Defect, Hawk-Dove → the 0.5 interior ESS, Rock-Paper-Scissors → an orbit around the centre that never converges, and Stag Hunt → all-Stag or all-Hare depending on the start. The non-converging RPS orbit is why populations in cyclic games need explicit machinery to avoid spinning.

10.4 The spinning top — transitive vs cyclic structure

If cyclic games are the problem, we need to *measure* how cyclic a game is. The **spinning-top decomposition** splits a game’s payoff matrix into a **transitive** component (a skill ladder, captured by per-strategy ratings) and a **cyclic** component (what is left over — the rock-paper-scissors part).⁹ The name comes from the shape Czarnecki et al. found in real games: they are widest (most cyclic) among strategies of intermediate strength and narrow towards the extremes of skill.⁴

⁹Balduzzi, D., Tuyls, K., Pérolat, J. & Graepel, T. (2018). “Re-evaluating Evaluation.” *NeurIPS*; arXiv:1806.02643 — the Hodge decomposition of an antisymmetric payoff matrix into transitive and cyclic components; Nash averaging.



same game, different populations: a best-response population (PSRO) sits in the wide cyclic belly; a training-snapshot population (league) climbs the transitive spine.

Figure 3: The spinning top (schematic): a vertical transitive axis (skill — there is a better player) and a cyclic dimension (width — the number of counters, rock-paper-scissors structure). Rock-Paper-Scissors sits in the widest cyclic belly (transitive ratio 0.0); a pure skill ladder sits on the transitive spine (1.0); the PSRO-Leduc best-response meta-game sits in the wide cyclic belly (~ 0.45 transitive), while the league’s snapshot meta-game climbs the transitive spine ($\sim 0.94-0.98$). T = transitive ratio, C = cyclic ratio.

A subtlety worth flagging: the original plan suggested an **SVD rank-1** decomposition, which wrongly reports Rock-Paper-Scissors as ≈ 0.707 transitive — the singular values of an antisymmetric matrix come in equal pairs, so a rank-1 truncation captures at most half of its squared norm for *any* game (a pure skill ladder also scores 0.707). The implementation uses the **combinatorial-Hodge** (ratings-difference) method instead,⁹ which correctly gives RPS a transitive ratio of 0.0. Measured:

Table 3: Transitive and cyclic components of four populations, by combinatorial-Hodge decomposition.

Population	Transitive (Hodge)	Cyclic	Structure
Rock-Paper-Scissors	0.0	1.0	purely cyclic
Pure skill ladder	1.0	0.0	purely transitive
PSRO-Leduc	0.41-0.46	0.89-0.91	mostly cyclic (27 three-cycles)
best-response meta-game			
League snapshot meta-game	0.94-0.98	—	mostly transitive

The two *real* populations are the headline — and they disagree, on the same game.¹⁰

Reconciliation (kept prediction → what actually happened). I framed poker as a skill ladder and expected Leduc’s meta-game to be mostly transitive. Measured, it depends entirely on *which population you decompose*. A population of **best responses** (PSRO) is mostly **cyclic** (≈ 0.45 transitive, 27 three-cycles): the best response beats the current mixture, a newer best response beats *that*, and so on — Czarnecki et al.’s spinning top in action. A population of **training-trajectory snapshots** (the league) is mostly **transitive** (≈ 0.94 - 0.98), because later snapshots are usually stronger than earlier ones, forming a ladder. Neither is a bug; the transitive/cyclic ratio is a property of the *population*, and choosing how you build the population is choosing whether you see a wheel or a ladder.

¹⁰Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML* — the transitive/cyclic decomposition of functional-form games; $\text{PSRO}_{r,N}$.

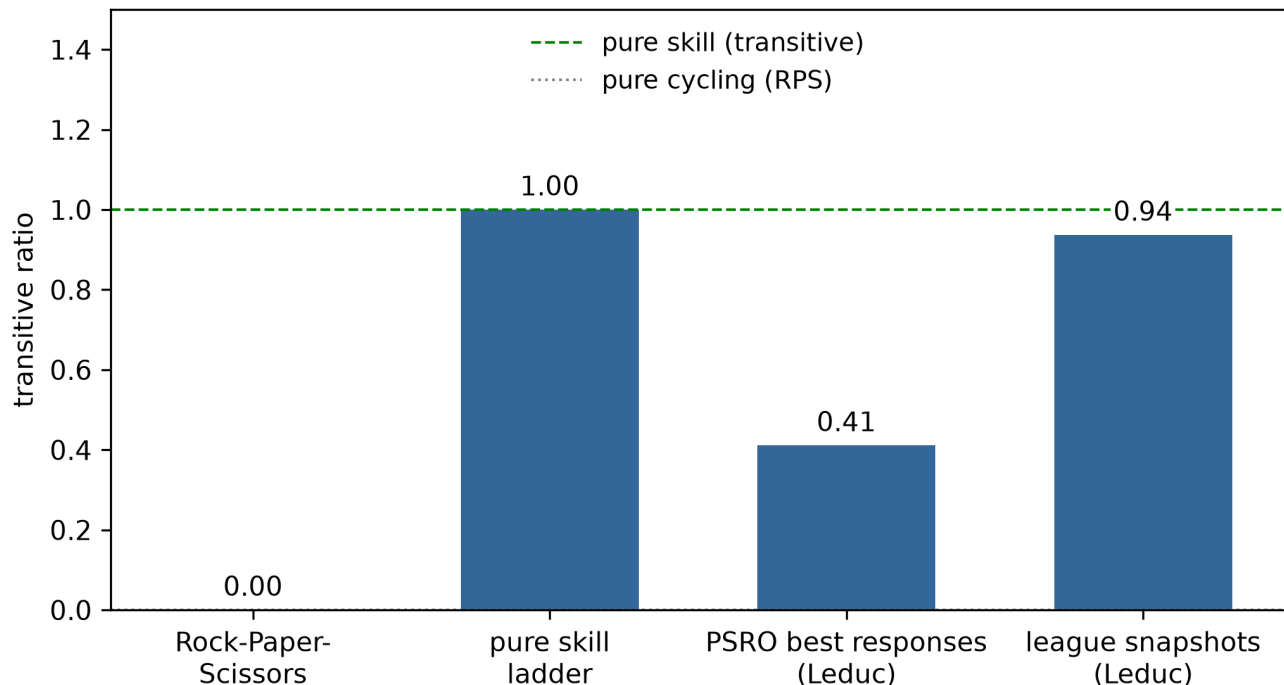


Figure 4: Transitive ratio across four populations: RPS (0.0, purely cyclic), a pure skill ladder (1.0), the PSRO-Leduc best-response meta-game (~ 0.41 - 0.46 , mostly cyclic), and the league snapshot meta-game (~ 0.94 - 0.98 , mostly transitive). Same game, opposite structure, depending on how the population is built.

10.5 The AlphaStar-style PBT league

With the diagnostic in hand, we build the dojo. The league trains neural PPO agents on **Leduc Hold'em** with three agent types: **main** agents (the product; they play everyone via PFSP self-play), **main exploiters** (hunt weaknesses in the *current* mains), and **league exploiters** (hunt weaknesses anywhere in the *frozen history*). Population-based training copies the top agents (exploit) and perturbs their learning rate / entropy (explore); periodically a frozen snapshot of each agent is added to the museum; **PFSP** (prioritized fictitious self-play) matchmaking samples each opponent with probability rising in how hard it is to beat. Critically, every neural network is extracted to a **tabular** policy so Chapter 7's **exact** best response measures its exploitability — the same NashConv used since Chapter 2 (NashConv is the sum of the two best-response gains; in OpenSpiel's convention, exploitability is half of it).

PFSP matchmaking: each opponent is sampled with a probability that rises with how hard it is to beat (training focuses where the agent is losing).

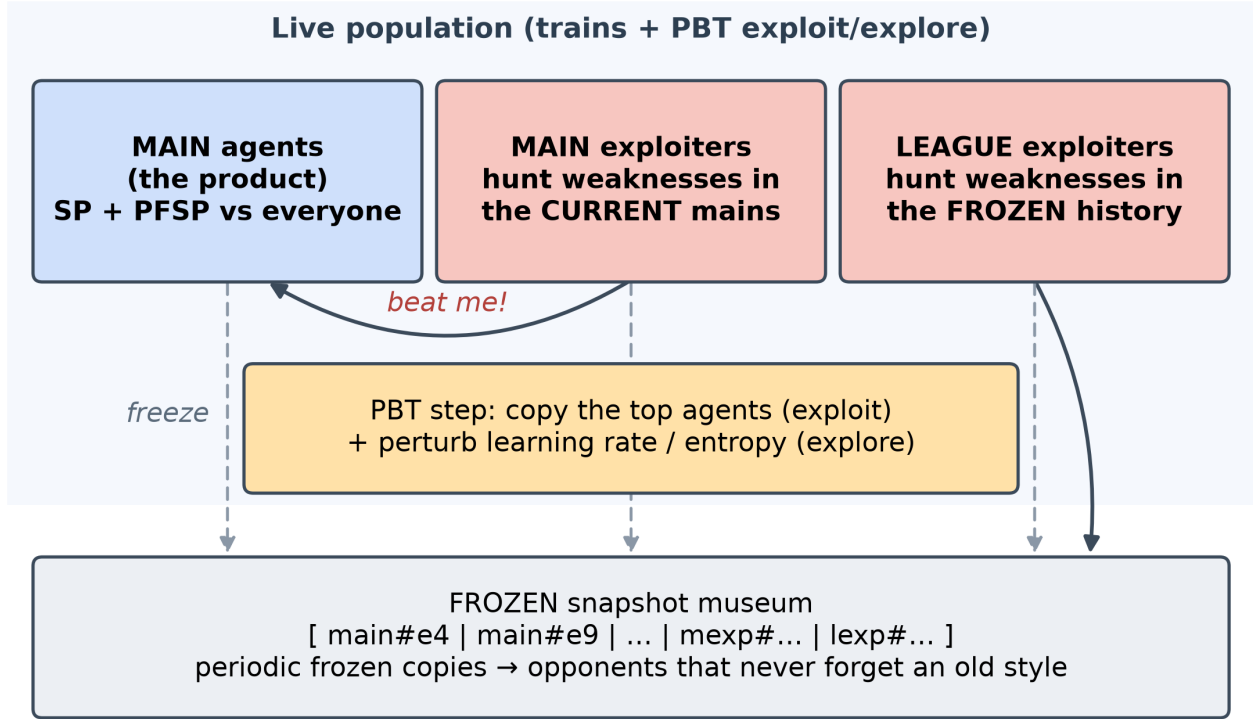


Figure 5: The league: main agents (the product), main exploiters (hunt weaknesses in the current mains), and league exploiters (hunt weaknesses in the frozen history), with periodic freezing into a snapshot museum and PFSP matchmaking that focuses training where the agent is losing. PBT copies the top agents and perturbs their hyper-parameters.

Does it improve? Measured, over two configs (smoke: 7 agents, 15 epochs; scale: 8 agents, 120 epochs, 48 frozen snapshots):

Table 4: League training metrics at both scales.

Metric	Smoke	Scale
min-main exploitability	4.67 → 3.04 (ends at min)	4.73 → min ≈ 1.21 → 2.05
meta-Nash exploitability	4.73 → 3.04	5.01 → min ≈ 1.32 (plateau 1.60) → 2.96
final Elo (live agents)	1176-1211	1198-1210

(The 2.96 is the meta-Nash of the population at the start of epoch 119, before its last training step; the 3.418 in §10.7 is the final 56-agent population.)

Reconciliation (kept prediction → what actually happened). I predicted a *monotone* decrease in exploitability. Smoke’s 15 epochs oblige — a clean drop that ends at its minimum. But scale’s 120 epochs tell the real story: exploitability falls steeply within ~ 15 epochs and is lowest around epochs 20–40 (meta-Nash ≈ 1.32 at epoch 21; best snapshot from epoch 29; the single lowest min-main value, ≈ 1.21 , at epoch 66), then holds a ≈ 1.60 meta-Nash plateau, and then **regresses** back up to ≈ 2.05 / ≈ 2.96 by epoch 119. The best agents are the *frozen snapshots* from the first half of the run; the *live* main agents get worse late (churn / partial forgetting). Self-play, which has no exploiters, also degrades after epoch 100, so exploiter pressure is not the only explanation. This is only visible once training is long enough — a running league is not a monotonically improving one. The remedy (untested here) is best-snapshot retention / population regularization. Methodologically it echoes Chapter 9: **scale reveals what smoke hides**.

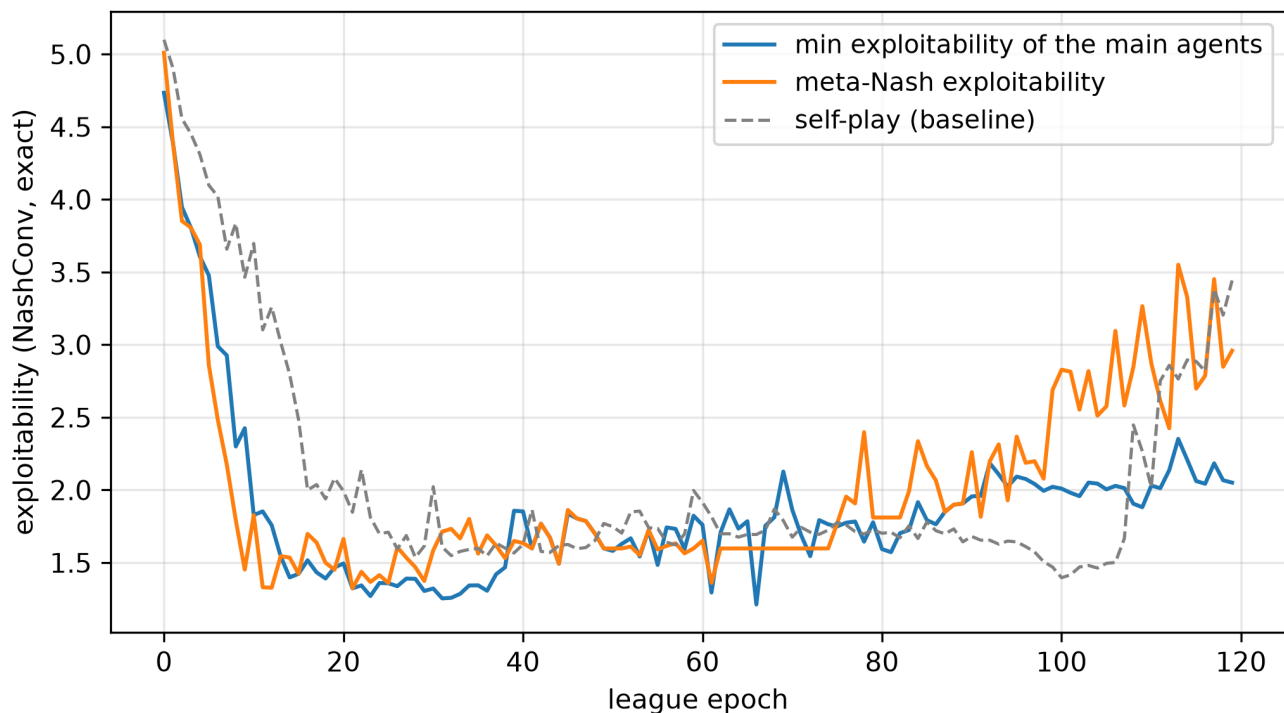


Figure 6: League exploitability (NashConv) over 120 epochs (scale): the minimum over the main agents and the meta-Nash exploitability fall steeply within the first ~ 15 epochs, are lowest around epochs 20–40, and rise again late in training. The frozen snapshots keep the strong agents of the first half of the run; the live agents degrade late. The dashed grey line is self-play (baseline), which also degrades after epoch 100.

10.6 Diversity — is the population actually diverse?

A league is only as good as the *variety* of strategies it holds. We measure three things: the **effective population size** (participation ratio of the meta-Nash weights), **behavioral clustering** (are the policies actually different?), and **exploit coverage**. Measured, the population is only weakly diverse: participation ratio rises from 1.0 (smoke) to 1.9 (scale), but behavioral clustering collapses everything into a **single cluster** at both scales — even at scale, where the maximum pairwise behavioral distance (0.48) exceeds the clustering threshold (0.30), single-linkage merges the chain. Diversity here is *weight-level* (the meta-Nash spreads support over a few agents), not *behavior-level* (the agents play near-identically).

The mechanism is clearest on a fast toy — a mini-PBT on matrix games:

Table 5: Mini-PBT diversity over generations, on a transitive and on a cyclic game.

Game	Diversity over generations
Prisoner’s Dilemma (transitive)	collapses to 0 (everyone $\rightarrow$ Defect)
Rock-Paper-Scissors (cyclic)	churns forever (0.07-0.29), never settles

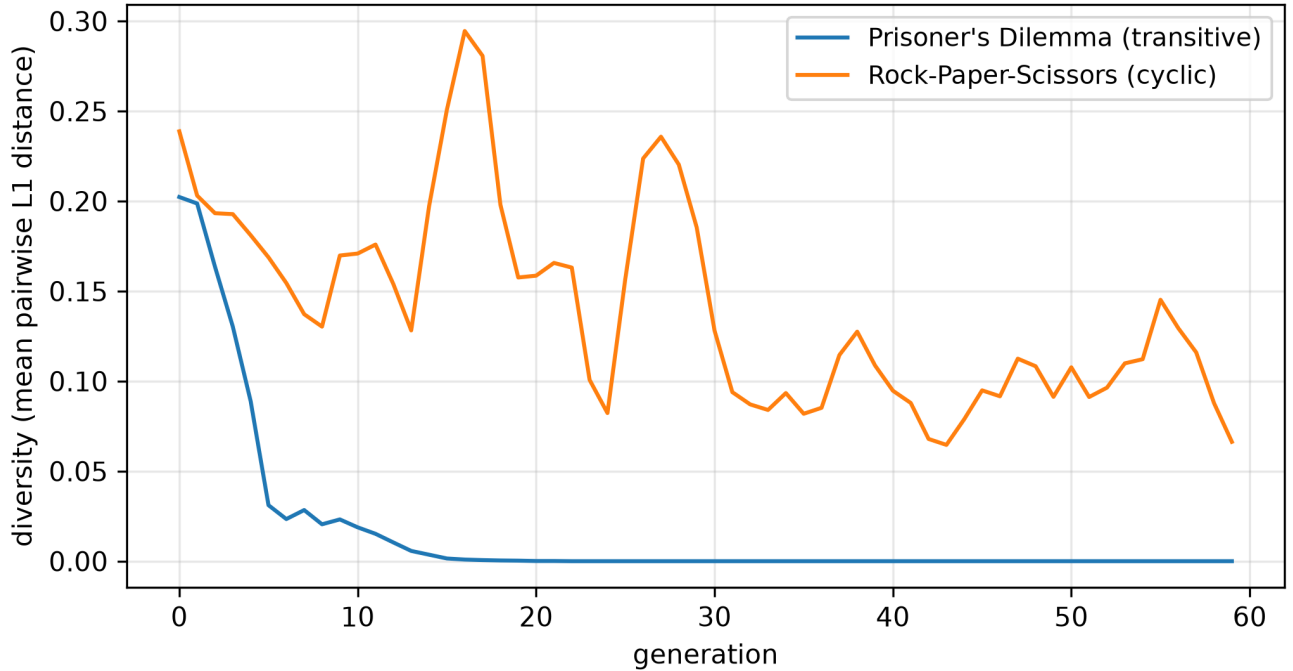


Figure 7: Mini-PBT diversity: on the transitive Prisoner’s Dilemma the population collapses to a single strategy (diversity $\rightarrow$ 0), while on cyclic Rock-Paper-Scissors it churns indefinitely (0.07-0.29) as it chases the wheel of counters. Game structure decides whether diversity survives.

Transitive games *kill* diversity (there is one best, everyone converges to it); cyclic games *force* it (there

is no best, so the population keeps churning). The Leduc league sits near the transitive end, which is exactly why its diversity is thin — and why the diversity benefits reported for AlphaStar, whose league created almost 900 distinct players, do not materialize at this scale.¹

10.7 EGTA — evaluating the population, and a surprise

The last tool is **empirical game-theoretic analysis (EGTA)**: treat whole policies as the “strategies” of a higher game, play every pair to fill an empirical payoff matrix, solve its **meta-Nash** mixture, and score it. It is the population lift of exploitability — Nash of a game whose atoms are policies.

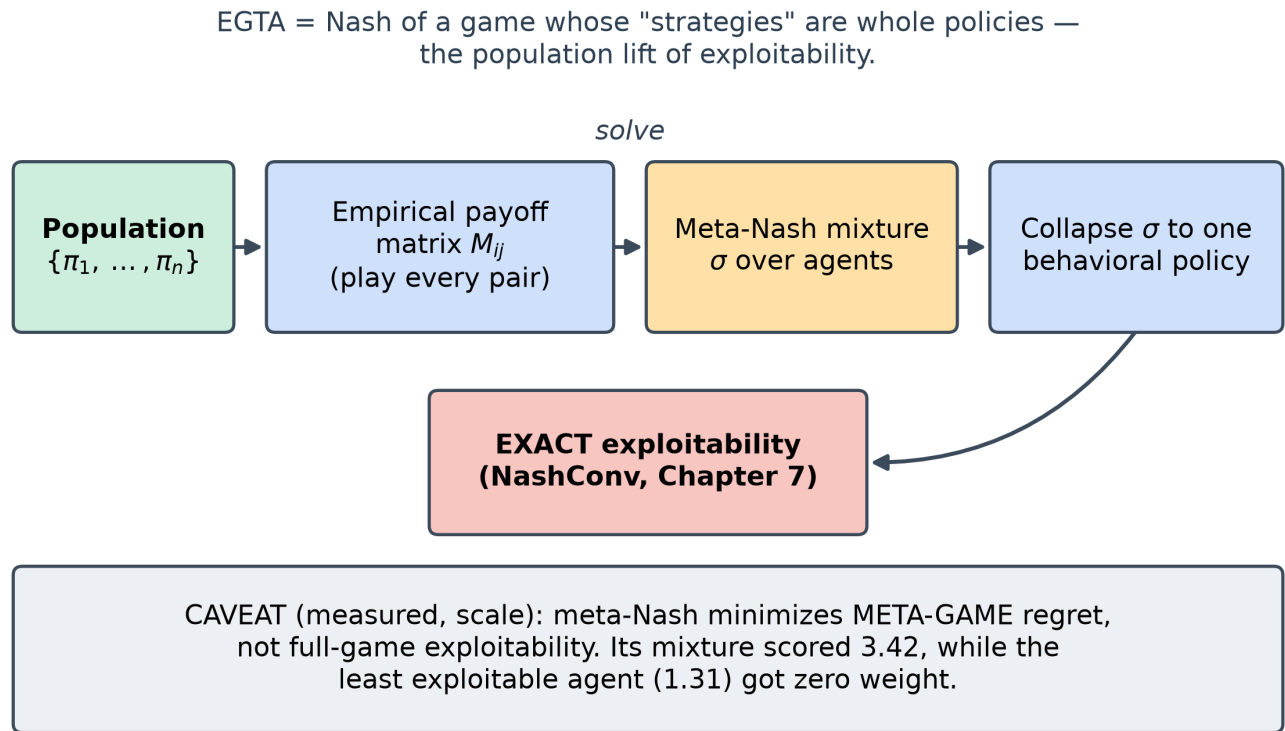


Figure 8: The EGTA pipeline: play every pair of agents to build an empirical payoff matrix, solve its meta-Nash mixture, collapse the mixture to a single behavioral policy, and measure its EXACT full-game exploitability. The measured caveat: the meta-Nash minimizes meta-game regret, not full-game exploitability, so the mixture can be more exploitable than the population’s best member, which may not even be in it.

The expectation (from the original plan) was that the meta-Nash of the league would be *less* exploitable than any single member. Measured:

Table 6: Meta-Nash exploitability against the best individual agent, per configuration.

Config	meta-Nash exploitability	best individual	meta-Nash $\leq$ best?
Smoke	2.665	2.665	yes (all weight on the best agent)
Scale	3.418	1.305	no

Reconciliation (kept prediction $\rightarrow$ what actually happened). I expected the meta-Nash mixture to be at least as unexploitable as the population’s best member. Smoke confirmed it trivially — the meta-Nash put *all* weight on the single best agent, so meta = best = 2.665. At scale the meta-Nash spreads weight (0.645 on one agent, smaller weights on two others), and the collapsed behavioral mixture scores 3.418 — **worse** than the best single snapshot (1.305). This is not a mixing bug: the identical code path gave meta = best in smoke, and the mixture is in fact *less* exploitable than each of its three components (3.558, 3.932, 4.262). The cause is the objective: the meta-Nash minimizes **meta-game regret** — doing well *against the population* — which is a *different objective* from minimizing **full-game exploitability**, so the least exploitable agents simply received zero meta-Nash weight. The lesson inverts: what a league should *ship* is a selected, best-response-robust member — not the meta-Nash mixture.

How does the league compare to the alternatives on Leduc exploitability?

Table 7: Leduc exploitability (NashConv) by method; for self-play both the final agent and the best iterate are shown.

Method (scale)	Exploitability (NashConv)
CFR-Nash (the floor)	0.0099
League — best individual (best of the run)	1.305
Self-play — best iterate (epoch 100)	1.396
PSRO (exact best-response oracle)	2.163
League — meta-Nash mixture	3.418
Self-play — final agent	3.683

The league’s **best individual** (1.305, the best snapshot of the whole run) is the best learned result, but its lead needs a careful reading: PSRO reaches 2.163, and self-play’s best iterate is 1.396 (epoch 100) before it degrades to 3.683 at the end — one run each. All of these remain far above the CFR-Nash floor. The league’s **mixture** (3.418) is better than the final self-play agent but worse than PSRO and the best individual (see the reconciliation above). Ship a selected member, not the mixture.³

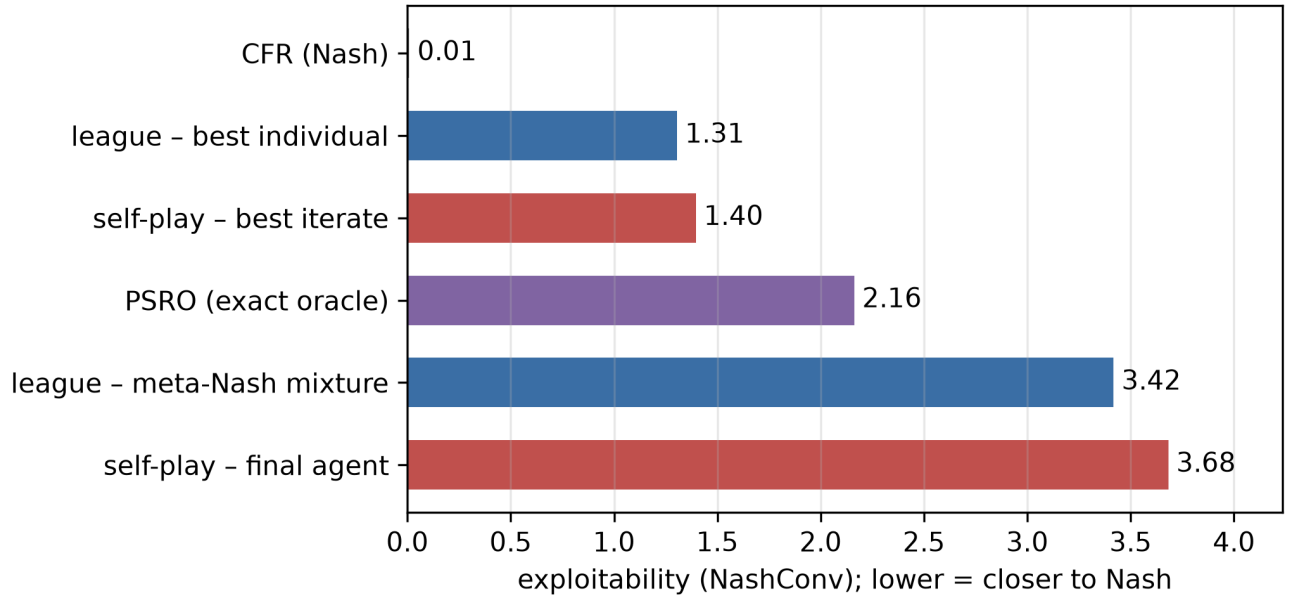


Figure 9: Leduc exploitability (NashConv) by method (scale): CFR-Nash floor ~ 0.01 ; the league’s best individual (1.31, best of the whole run) is comparable to self-play’s best iterate (1.40) and below PSRO (2.16), the league’s meta-Nash mixture (3.42) and the final self-play agent (3.68).

10.8 Honest notes, limitations, and where this hands off

What held up. The confirmed backbone: replicator dynamics reproduce every analytic outcome of the matrix games (ESS or Nash equilibrium), including RPS’s closed orbit that never converges;⁷ the Hodge spinning-top decomposition cleanly separates skill from cycles and correctly labels the pure cases; the league drives strong *early* improvement and produces an individual agent at 1.305 — better than PSRO (2.163) and comparable to self-play’s best iterate (1.396); and EGTA gives a working population-level exploitability. Together they trace the arc from “which games can a population even settle in?” to “what happens when a population trains itself?”

What did not, and why it matters. Three honest caveats travel forward. (1) The league’s exploitability **regresses late** at scale ($4.73 \rightarrow \approx 1.21 \rightarrow \approx 2.05$) — a running league is not a monotonically improving one; the best agents are frozen snapshots. (2) The **meta-Nash mixture is more exploitable than the population’s best member** at scale (3.42 vs 1.31), because it gives that member no weight — choosing by meta-Nash does not guarantee robustness, so ship a selected member, not the mixture. (3) Leduc’s meta-game is **cyclic as a best-response population** but **transitive as a snapshot population** — structure is a property of the population you build. And a methodological point: the late regression and the mixture’s weakness are both invisible at the fast smoke config — scale reveals what smoke hides.

Trust. Every evolutionary target is *analytic* (ESS/Nash of the matrix games), and every league exploitability is Chapter 7’s *exact* NashConv on tabular-extracted policies — so the game-theoretic results

are bounded by ground truth. The neural results are a single PBT run per config: the *directions* (early improvement; best snapshot < PSRO < final self-play agent; late regression; meta-Nash mixture > the population’s best member) are the trustworthy claims, not the third-decimal magnitudes. The experiment plots above are drawn from the saved JSON result files; the conceptual diagrams are generated by the `make*_figure.py` scripts.

Backward and forward connections. Backward: the league is Chapter 9’s PSRO made asynchronous with neural oracles, and it reuses Chapter 7’s exact best response wholesale; the “meta-Nash of a population” is Chapter 2’s Nash lifted one level. Forward: main exploiters are automated opponent modelers (Contribution #1); EGTA/meta-Nash is the evaluation methodology (Contribution #3); and the league’s missing guarantee — it can regress and its mixture can be exploitable — is the population form of the **missing $N > 2$ safety anchor** (Contribution #2). The transitive/cyclic diagnostic predicts Chapter 11’s FFA coalition games will be strongly cyclic, so naive PBT there will cycle.

10.9 Key takeaways for the thesis synthesis

- **Structure decides convergence.** The transitive/cyclic (spinning-top) ratio is a *pre-training diagnostic*: transitive games settle (and kill diversity), cyclic games spin (and force diversity). Measured: RPS 0.0 transitive, skill ladder 1.0, PSRO-Leduc ≈ 0.45 (cyclic), league snapshots ≈ 0.94 -0.98 (transitive).
- **The league produces strong individuals but carries no guarantee.** Best snapshot 1.305 beats PSRO 2.163 and matches self-play’s best iterate (1.396; its final agent is 3.683) — one run each — yet exploitability *regresses* late ($\rightarrow 2.05$) and the meta-Nash *mixture* (3.42) is worse than the population’s best member (1.31).
- **Meta-Nash optimizes meta-game regret, not full-game exploitability** — the two disagree, so population evaluation must score the collapsed mixture directly and ship a selected member.
- **Exact evaluation is the anchor.** Extracting neural policies to tabular and grading them with Chapter 7’s exact best response makes every population claim ground-truthed — the same NashConv since Chapter 2.
- **The population-safety gap is the open door (Contribution #2):** the AlphaStar exploiter mechanism is the population analog of Chapter 8’s safe exploitation, but it is heuristic — its robustness is shown only empirically,¹ and in this chapter’s small league it guaranteed neither monotone improvement nor a non-exploitable mixture.